

HACKATHON CHALLENGE BRIEF

Touchless Invoice Agent (TIA)

Build an AI agent that turns timesheets into dispatched invoices — touchlessly

1. Background & Context

In services businesses, billing a client is surprisingly manual. Each month, timesheets arrive from many sources and in many formats — clean Excel sheets, scanned PDFs, and even photos of handwritten attendance registers. Finance teams manually clean and consolidate this data, feed it into an ERP (e.g., SAP) to run payroll, generate invoices, check each invoice against business rules, sort them, and finally email them to clients. Every step is slow, error-prone, and hard to track.

Your challenge is to design and prototype a Touchless Invoice Agent (TIA): an AI-driven system that takes timesheet inputs from any channel and any format and carries them all the way to validated, dispatched invoices — with humans stepping in only when needed. “Touchless” is the goal: the happy path should require zero manual intervention, while still keeping a human in the loop for exceptions.

This is a realistic, end-to-end agentic-AI problem. It rewards teams that combine document AI (OCR / handwriting recognition), workflow orchestration, sensible UX, and trustworthy AI practices (accuracy indicators and human oversight).

2. The Problem Statement

Build a Touchless Invoice Agent that provides a touchless experience for users to dispatch invoices starting from timesheet input.

The agent should ingest timesheets from multiple channels, convert mixed-format data (including handwritten documents) into a single ERP-uploadable format, orchestrate invoice generation, validate the invoices against business rules, dispatch them according to client-specific rules, and give every stakeholder transparency through dashboards and an AI assistant. Three personas interact with the system:

Persona	Who they are	What they need
Client	The customer being billed.	Submit timesheets, review and approve invoices, raise queries.
FinOps Team	Operations team running the billing process.	Configure clients & rules, handle exceptions, resolve client queries.
Finance Team	Owners of financial accuracy & compliance.	Ensure invoices are correct and validated; analytics and reporting.

3. Reference Solution Shape

This diagram illustrates one valid way to structure the solution. It is a guide, not a prescription — teams are free to innovate on architecture, models, and tooling.

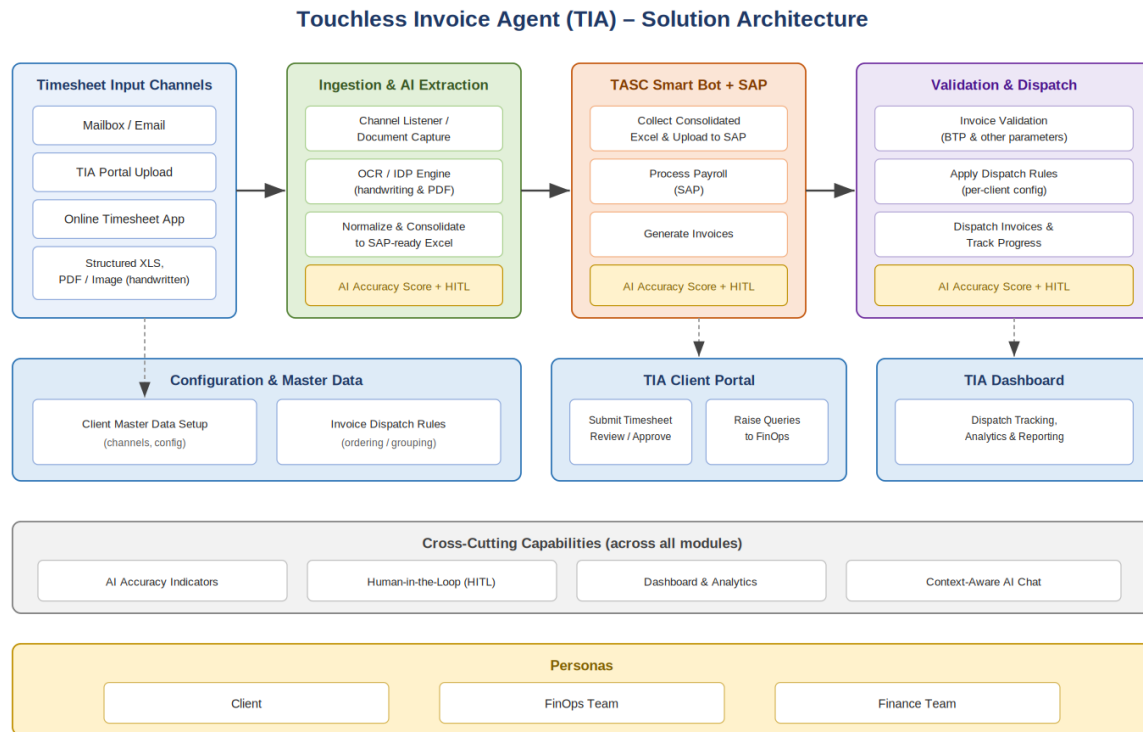


Figure 1 — Reference architecture & end-to-end flow

4. What to Build (Functional Challenges)

Tackle as many of the following modules as your time allows. Depth on a few well-executed modules beats shallow coverage of all.

4.1 Client Configuration & Master Data

- A setup screen to onboard a client and configure its input channels and invoice dispatch rules.
- Per-client parameters that drive downstream processing (currency, billing entity, validation profile).

4.2 Multi-Channel Timesheet Ingestion

- Read timesheets from at least two channels (e.g., a mailbox/email and a portal upload; bonus for an online timesheet app).
- Accept both structured Excel files and unstructured PDFs/images, including handwritten data.

4.3 AI Extraction & ERP-Ready Conversion (core challenge)

- Use OCR / Intelligent Document Processing to extract data from PDFs and handwritten images.
- Normalize and consolidate everything into a single ERP-uploadable (e.g., SAP-style) Excel format.
- Show an AI accuracy/confidence score per document and allow human review before release.

4.4 Invoice Generation Orchestration

- Simulate or integrate a bot/ERP step that takes the consolidated file, “processes payroll,” and generates invoices.
- A stub/mock ERP is acceptable — the focus is the orchestration and data flow, not a real SAP instance.

4.5 Invoice Validation

- Validate each generated invoice against a configurable rule set (define your own “BTP-style” parameters).
- Flag failures and route them to a human-in-the-loop review.

4.6 Dispatch Rules & Tracking

- A configuration screen to define per-client dispatch rules (e.g., order bills ascending by spend amount or salary).
- Dispatch invoices per the rule and track progress/status on a dashboard.

4.7 Client Portal

- Let clients submit timesheets, review and approve invoices, and raise queries for FinOps to answer.

4.8 Cross-Cutting Capabilities (judged across all modules)

- AI accuracy indicators shown across timesheets, invoices, and dispatch.

- Human-in-the-loop controls available throughout.
- Dashboard & analytics for operations and finance.
- A context-aware AI chat assistant that understands the current client/invoice/timesheet context.

5. Minimum Viable Demo vs. Stretch Goals

	Minimum Viable Demo	Stretch Goals
Ingestion	One channel + Excel input.	Email + portal + handwritten image OCR.
Extraction	Parse structured Excel to a clean format.	Accurate handwriting/PDF extraction with confidence scores.
Generation	Mock invoice generation from consolidated data.	Realistic orchestration with status & exception capture.
Validation	Basic rule checks (totals, required fields).	Configurable rule engine + auto-routing of failures.
Dispatch	List invoices in a configured order.	Rule-config screen + dispatch tracking dashboard.
AI / UX	Accuracy score on extraction.	Full HITL + context-aware AI chat assistant.

6. Evaluation Criteria

Submissions will be judged on the following weighted criteria:

Criterion	Weight	What judges look for
End-to-end “touchless” flow	25%	A working path from timesheet input to dispatched invoice.
AI / document intelligence	20%	Quality of extraction, especially handwritten/PDF, with accuracy indicators.
Trust & human-in-the-loop	15%	Sensible review checkpoints; graceful handling of low-confidence cases.
UX & dashboards	15%	Clarity of portal, config screens, dashboard, and AI chat.
Technical execution	15%	Architecture, code quality, robustness of the demo.
Innovation & presentation	10%	Originality and a clear, compelling demo/pitch.

7. Deliverables

1. A working prototype (web app and/or backend) demonstrating the end-to-end flow.
2. A public code repository (GitHub/GitLab) with a clear README and setup steps.
3. A short demo video (3–5 minutes) or live demo walking through the touchless flow.

4. A brief pitch deck (max 8 slides): problem, approach, architecture, demo, and what's next.
5. Sample input data used (Excel, a PDF, and at least one handwritten sample).

8. Suggested Tools & Tech (optional)

Use anything you like. These are starting points, not requirements:

- Frontend: React / Next.js, or any framework you are comfortable with.
- Backend: Node.js/Express, Python/FastAPI, or similar.
- Document AI / OCR: open-source OCR (e.g., Tesseract), cloud document-AI services, or vision-capable LLMs for handwriting.
- AI assistant: any LLM API for the context-aware chat and extraction reasoning.
- Data/ERP: mock ERP with a simple service or spreadsheet; a real ERP is not required.

9. Tips to Win

- Nail one complete vertical slice first (e.g., Excel → invoice → dispatch), then add channels and handwriting.
- Treat the handwritten-document extraction as the hardest and most differentiating part — show your confidence handling.
- Make demo tell a story across the three personas; judges remember a clear narrative.
- Don't hide the AI — surface accuracy scores and human-in-the-loop moments; trust is part of the score.

10. FAQ

Question	Answer
Do we need a real SAP system?	No. Mock or stub the ERP. We care about the flow, data handling, and AI.
What is "BTP" in validation?	Treat it as a placeholder for a configurable business validation parameter set — define your own.
Must we build all modules?	No. Depth and a working end-to-end slice beat broad but shallow coverage.
Can we use commercial AI APIs?	Yes, if you can run the demo. Have a fallback in case of connectivity limits.